

NPS Experiment 012 — Policy Subspace Causal Necessity (Probe-Ablation Test)

Neural Privilege Separation (NPS) research series

Research question. Experiment 011 found that `Qwen/Qwen2.5-1.5B-Instruct` encodes benign-vs-unsafe policy information in a low-dimensional subspace (intrinsic dimensionality ~14-35 components at 90% variance depending on layer, U-shaped across depth), and that — once within-class topic variance is controlled for via between-class (paired-difference) PCA — the class-separating direction increasingly aligns with PC1 at depth. This notebook asks the natural follow-up:

Is that subspace causally necessary for the *linear separability* of policy-relevant activations — or is the same information redundantly encoded elsewhere in the residual stream, such that removing the subspace doesn't actually destroy it?

Scope, deliberately narrow. This is a **representational** causal test, not a behavioral one. We do not generate text, evaluate jailbreak success, or measure refusal rate — this notebook only asks whether *linear probes* can still separate benign/unsafe activations after the discovered subspace is projected out. That's a meaningfully different (and narrower) claim than "the model can be made to comply with unsafe requests," and the two should not be conflated when interpreting results below.

Method, in one paragraph. For each representative layer (early, middle, final) and each subspace definition from Experiment 011 (within-class PCA, between-class/paired-difference PCA), we project the top- k components out of held-out benign/unsafe activations for $k \in \{1, 2, 5, 10, 20\}$, then measure two things: (a) whether the **original, frozen** Experiment-011 probe still classifies the ablated activations correctly ("did we destroy the direction the probe was actually using?"), and (b) whether a **freshly re-fit** probe trained on the ablated activations can still separate the classes ("is the information gone, or just moved / redundantly encoded?"). Both are compared against ablating a random k -dimensional subspace of the same size, so we can tell subspace-specific effects from generic

dimensionality-reduction effects.

0. Runtime check

No text generation happens in this notebook, so a GPU is optional — it is only used as a fallback if some Experiment 011 activation cache is missing and needs to be (re-)extracted. If you're confident all caches are present, you can run this notebook on CPU-only Colab.

```
1 !nvidia-smi --query-gpu=name,memory.total,memory.used,driver_ve
2 import torch
3 print("CUDA available:", torch.cuda.is_available())
4
```

```
name, memory.total [MiB], memory.used [MiB], driver_version
Tesla T4, 15360 MiB, 0 MiB, 580.82.07
CUDA available: True
```

1. Setup

Installs pinned dependencies (matplotlib pinned to a specific stable release, matching the convention adopted in the Experiment 011 update) and mounts Google Drive. We resolve **two** base directories: this experiment's own `BASE_DIR` (for its own resumable outputs) and `PRIOR_BASE_DIR`, pointing at Experiment 011's results — that's where cached activations, probes, and PCA subspaces are reused from.

```
1 %%capture
2 # matplotlib pinned to 3.8.4 / seaborn to 0.13.2 – same stable
3 # Experiment 011 notebook update, avoiding intermittent Colab r
4 # seen on newer matplotlib releases.
5 !pip install -q -U "transformers>=4.46" "accelerate>=0.34" "sci
6     "matplotlib==3.8.4" "seaborn==0.13.2" "pandas>=2.2" "scipy>
7
```

```

1 from google.colab import files
2 uploaded = files.upload() # pick nps_exp011_policy_subspace_res
3
4 import zipfile, os
5 os.makedirs('/content/nps_exp011_policy_subspace/results', exist
6 with zipfile.ZipFile('nps_exp011_policy_subspace_results-3.zip')
7     z.extractall('/content/nps_exp011_policy_subspace/results')

```

Choose Files  nps_exp011_...ults-3.zip

nps_exp011_policy_subspace_results-3.zip(application/zip) - 20207957 bytes, last modified: n/a - 100% done

```

1 import os
2
3 USE_DRIVE = False # set False to keep everything local to the
4
5 if USE_DRIVE:
6     try:
7         from google.colab import drive
8         drive.mount('/content/drive')
9         DRIVE_NPS_ROOT = '/content/drive/MyDrive/NPS'
10    except Exception as e:
11        print("Drive mount failed or not in Colab, falling back
12        DRIVE_NPS_ROOT = None
13 else:
14     DRIVE_NPS_ROOT = None
15
16 # --- this experiment's own resumable output directory ---
17 BASE_DIR = (os.path.join(DRIVE_NPS_ROOT, 'exp012_policy_subspac
18             if DRIVE_NPS_ROOT else '/content/nps_exp012_policy_
19 RESULTS_DIR = os.path.join(BASE_DIR, 'results')
20 ABLATION_DIR = os.path.join(RERESULTS_DIR, 'ablation')
21 STATS_DIR = os.path.join(RERESULTS_DIR, 'statistics')
22 FIGURES_DIR = os.path.join(RERESULTS_DIR, 'figures')
23 for d in [RESULTS_DIR, ABLATION_DIR, STATS_DIR, FIGURES_DIR]:
24     os.makedirs(d, exist_ok=True)
25
26 # --- Experiment 011's output directory, where we reuse activat
27 PRIOR_RESULTS_DIR_OVERRIDE = None # set explicitly if your Exp
28
29 def resolve_prior_results_dir():
30     if PRIOR_RESULTS_DIR_OVERRIDE:
31         return PRIOR_RESULTS_DIR_OVERRIDE
32     candidates = []
33     if DRIVE_NPS_ROOT:
34         candidates.append(os.path.join(DRIVE_NPS_ROOT, 'exp011_
35         candidates.append('/content/nps_exp011_policy_subspace/resu

```

```

36     for c in candidates:
37         if os.path.isdir(c) and os.path.isdir(os.path.join(c, '
38             return c
39     raise FileNotFoundError(
40         "Could not locate Experiment 011 results (looked in: "
41         f"{candidates}). Set PRIOR_RESULTS_DIR_OVERRIDE to the
42         "folder (e.g. a Drive path) before continuing."
43     )
44
45 PRIOR_RESULTS_DIR = resolve_prior_results_dir()
46 PRIOR_ACTS_DIR     = os.path.join(PRIOR_RESULTS_DIR, 'activatic
47 PRIOR_PROBES_DIR   = os.path.join(PRIOR_RESULTS_DIR, 'probes')
48 PRIOR_SUBSPACE_DIR = os.path.join(PRIOR_RESULTS_DIR, 'subspace'
49
50 print("This experiment's outputs:", RESULTS_DIR)
51 print("Reusing Experiment 011 artifacts from:", PRIOR_RESULTS_D
52

```

This experiment's outputs: /content/nps_exp012_policy_subspace_inter
 Reusing Experiment 011 artifacts from: /content/nps_exp011_policy_su

✓ 2. Configuration

`seed`, `test_size`, `probe_C`, and `probe_max_iter` are set to match Experiment 011 exactly — this lets us reproduce the *same* train/test split (via `train_test_split` with the same `random_state`) so the "frozen probe" evaluation is on the identical held-out set the original probe was scored on.

```

1 from dataclasses import dataclass, field
2 from typing import List
3 import json
4 import torch as _torch_probe
5
6 @dataclass
7 class Config:
8     model_name: str = "Qwen/Qwen2.5-1.5B-Instruct" # only used
9     dtype: str = "float16"
10    device: str = "cuda" if _torch_probe.cuda.is_available() el
11    seed: int = 42 # must match Exp011's config
12    test_size: float = 0.2 # must match Exp011's config
13    probe_C: float = 1.0 # must match Exp011's config
14    probe_max_iter: int = 2000 # must match Exp011's config
15
16    k_list: List[int] = field(default_factory=lambda: [1, 2, 5,
17    subspace_types: List[str] = field(default_factory=lambda: [
18    n_random_trials: int = 20 # random-subspace control tr
19    n_bootstrap: int = 2000 # bootstrap resamples for ac
20    ci_level: float = 0.95
21
22 config = Config()
23 print(json.dumps(config.__dict__, indent=2, default=str))
24

```

```

{
  "model_name": "Qwen/Qwen2.5-1.5B-Instruct",
  "dtype": "float16",
  "device": "cuda",
  "seed": 42,
  "test_size": 0.2,
  "probe_C": 1.0,
  "probe_max_iter": 2000,
  "k_list": [
    1,
    2,
    5,
    10,
    20
  ],
  "subspace_types": [
    "within",
    "between"
  ],
  "n_random_trials": 20,
  "n_bootstrap": 2000,
  "ci_level": 0.95
}

```

✓ 3. Stage & checkpoint utilities

Identical conventions to Experiment 011: a JSON stage manifest gates each major stage so re-running the notebook after a disconnect skips completed work; every stage saves its output immediately and then cleans up.

```

1 import gc, time, pickle, json, os
2 import numpy as np
3 import torch
4
5 STAGE_FILE = os.path.join(RESULTS_DIR, "stage_manifest.json")
6
7 def _load_manifest():
8     if os.path.exists(STAGE_FILE):
9         with open(STAGE_FILE) as f:
10             return json.load(f)
11     return {}
12
13 def _save_manifest(m):
14     with open(STAGE_FILE, "w") as f:
15         json.dump(m, f, indent=2)
16
17 def stage_done(name):
18     return _load_manifest().get(name, {}).get("done", False)
19
20 def mark_done(name, meta=None):
21     m = _load_manifest()
22     m[name] = {"done": True, "timestamp": time.time(), "meta":
23     _save_manifest(m)
24     print(f"[stage] '{name}' marked complete.")
25
26 def cleanup(*tensors):
27     for t in tensors:
28         try:
29             del t
30         except Exception:
31             pass
32     gc.collect()
33     if torch.cuda.is_available():
34         torch.cuda.empty_cache()
35
36 def save_pickle(obj, path):
37     tmp = path + ".tmp"
38     with open(tmp, "wb") as f:
39         pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)

```

```

40     os.replace(tmp, path)
41
42 def load_pickle(path):
43     with open(path, "rb") as f:
44         return pickle.load(f)
45
46 print("Stage manifest at:", STAGE_FILE)
47 print("Existing stages:", list(_load_manifest().keys()))
48

```

```

Stage manifest at: /content/nps_exp012_policy_subspace_intervention/
Existing stages: []

```

✓ 4. Load Experiment 011 artifacts

Reuses cached activations, trained probes, and both PCA subspace variants (within-class from Section 8, between-class/paired-difference from Section 8b) from Experiment 011. If an activation cache is missing for a layer we need, we fall back to re-extracting it with the same hook-based method — this is the *only* place this notebook touches the model or the GPU.

```

1 def act_path(layer, cls):
2     return os.path.join(PRIOR_ACTS_DIR, f"layer{layer:02d}_{cls}
3
4 def probe_path(layer):
5     return os.path.join(PRIOR_PROBES_DIR, f"probe_layer{layer:0
6
7 def within_pca_path(layer):
8     return os.path.join(PRIOR_SUBSPACE_DIR, f"pca_layer{layer:0
9
10 def between_pca_path(layer):
11     return os.path.join(PRIOR_SUBSPACE_DIR, f"pca_betweenclass_
12
13 with open(os.path.join(PRIOR_RESULTS_DIR, "stage_manifest.json"
14     prior_manifest = json.load(f)
15 prior_layers = sorted(prior_manifest.get("activation_extraction
16 print("Layers with cached activations in Experiment 011:", prior_layers)
17
18 with open(os.path.join(PRIOR_PROBES_DIR, "probe_metrics.json"))
19     prior_probe_metrics = json.load(f)
20
21 # Representative early/middle/final layers MUST match the speci
22 # Experiment 011 designated as "early"/"middle"/"final" (via ea
23 # middle_frac / final_frac) – these are NOT simply the min/max

```

```

24 # analyzed layer sweep, which instead spans the full `layer_fra
25 # (typically including layer 0 and the last layer, neither of w
26 # the same as the early/final *representative* layers). Reading
27 # `early_middle_final_comparison.csv` is the reliable source of
28 # fall back to recomputing from fracs if that file is missing.
29 emf_path = os.path.join(PRIOR_SUBSPACE_DIR, "early_middle_final
30 if os.path.exists(emf_path):
31     import pandas as _pd
32     _emf = _pd.read_csv(emf_path)
33     rep_layers = {row.regime: int(row.layer) for row in _emf.it
34 else:
35     num_layers = max(prior_layers) + 1
36     rep_layers = {
37         "early": int(round(0.15 * (num_layers - 1))),
38         "middle": int(round(0.50 * (num_layers - 1))),
39         "final": int(round(0.92 * (num_layers - 1))),
40     }
41     print("Warning: early_middle_final_comparison.csv not found
42           "recomputed rep_layers from default fracs – verify th
43           "Experiment 011's actual config if it used non-default
44 print("Representative layers for ablation study:", rep_layers)
45

```

Layers with cached activations in Experiment 011: [0, 3, 7, 10, 14,
Representative layers for ablation study: {'early': 4, 'middle': 14,

```

1 _model = None
2 _tokenizer = None
3
4 def get_model_and_tokenizer():
5     '''Lazy-load fallback, only used if an activation cache is
6     missing for one of the representative layers.'''
7     global _model, _tokenizer
8     if _model is not None:
9         return _model, _tokenizer
10    from transformers import AutoModelForCausalLM, AutoTokenizer
11    print(f"[fallback] Loading {config.model_name} to re-extra
12    _tokenizer = AutoTokenizer.from_pretrained(config.model_na
13    if _tokenizer.pad_token is None:
14        _tokenizer.pad_token = _tokenizer.eos_token
15    _model = AutoModelForCausalLM.from_pretrained(
16        config.model_name, torch_dtype=torch.float16, device_r
17    _model.eval()
18    return _model, _tokenizer
19
20 def extract_activations_fallback(prompts, cls, layer, batch_si
21     '''Minimal re-implementation of Exp011's extraction, used

```



```

22     required cache file is missing.'''
23     model, tokenizer = get_model_and_tokenizer()
24     captured = {}
25     def hook(module, inp, out):
26         hs = out[0] if isinstance(out, tuple) else out
27         captured["h"] = hs.detach()
28     h = model.model.layers[layer].register_forward_hook(hook)
29     buf = []
30     try:
31         for i in range(0, len(prompts), batch_size):
32             batch = prompts[i:i + batch_size]
33             msgs = [{"role": "user", "content": p} for p in
34             texts = [tokenizer.apply_chat_template(m, tokenize
35             enc = tokenizer(texts, return_tensors="pt", paddi
36             with torch.no_grad():
37                 model(**enc)
38                 attn = enc["attention_mask"]
39                 last_idx = attn.sum(dim=1) - 1
40                 pooled = captured["h"][torch.arange(captured["h"].
41                 buf.append(pooled)
42                 cleanup(enc)
43     finally:
44         h.remove()
45     arr = np.concatenate(buf, axis=0)
46     np.save(act_path(layer, cls), arr)
47     return arr
48
49 def load_layer_activations(layer):
50     b_path, u_path = act_path(layer, "benign"), act_path(layer
51     if not (os.path.exists(b_path) and os.path.exists(u_path))
52         raise FileNotFoundError(
53             f"Missing cached activations for layer {layer} and
54             f"to re-extract in this notebook. Point PRIOR_RESU
55             f"results folder that has them, or re-run Experime
56         )
57     return np.load(b_path), np.load(u_path)
58
59 def backfill_betweenclass_pca(layer, n_shuffles=5, seed=42):
60     '''Some Experiment 011 runs only produced within-class PCA
61     the between-class (paired-difference) variant. Rather than
62     reconstruct it here from the cached activations/probe that
63     same method as Experiment 011 Section 8b (uncentered SVD c
64     unsafe-benign difference vectors, several random pairings)
65     out_path = between_pca_path(layer)
66     if os.path.exists(out_path):
67         return
68     benign, unsafe = load_layer_activations(layer)
69     n = min(len(benign), len(unsafe))

```

```

70     rng = np.random.default_rng(seed + layer)
71     diffs = [unsafe[:n] - benign[rng.permutation(n)][:n] for _
72     D = np.concatenate(diffs, axis=0)
73     max_rank = min(128, D.shape[0], D.shape[1])
74     U, S, Vt = np.linalg.svd(D, full_matrices=False)
75     Vt, S = Vt[:max_rank], S[:max_rank]
76     evr = (S ** 2) / np.sum(S ** 2)
77     cum = np.cumsum(evr)
78     probe = load_pickle(probe_path(layer))
79
80     def _cos(a, b):
81         return float(np.dot(a, b) / (np.linalg.norm(a) * np.li
82
83     result = {
84         "layer": layer, "explained_variance_ratio": evr, "cumu
85         "variance_at_k": {k: float(cum[k - 1]) if k <= len(cur
86         "intrinsic_dim": {t: int(min(int(np.searchsorted(cum,
87         "components": Vt[:50], "pc1_vs_probe_cosine": _cos(Vt[
88         "pc1_vs_meandiff_cosine": _cos(Vt[0], D.mean(axis=0)),
89         "max_rank": max_rank, "n_diff_vectors": D.shape[0], "h
90     }
91     save_pickle(result, out_path)
92     cleanup(benign, unsafe, diffs, D, U, S, Vt, probe)
93     print(f"[backfill] Reconstructed missing between-class PCA
94
95 # Sanity check all representative layers are available before
96 # Activations, the frozen probe, and the within-class PCA are
97 # The between-class PCA is auto-backfilled if missing and "bet
98 for name, l in rep_layers.items():
99     if not (os.path.exists(act_path(l, "benign")) and os.path.
100         raise FileNotFoundError(f"Representative layer '{name}'
101     if not os.path.exists(probe_path(l)):
102         raise FileNotFoundError(f"Representative layer '{name}'
103     if not os.path.exists(within_pca_path(l)):
104         raise FileNotFoundError(f"Representative layer '{name}'
105     if "between" in config.subspace_types and not os.path.exis
106         backfill_betweenclass_pca(l)
107 print("All required Experiment 011 artifacts found for represe
108

```

```

[backfill] Reconstructed missing between-class PCA cache for layer 4
[backfill] Reconstructed missing between-class PCA cache for layer 1
[backfill] Reconstructed missing between-class PCA cache for layer 2
All required Experiment 011 artifacts found for representative layer

```

✓ 5. Ablation and evaluation machinery

- `project_out(H, V)` removes the span of the (orthonormal) rows of `V` from each row of `H`: $H' = H - (H \cdot V^T) \cdot V$.
- `eval_frozen_probe` applies the **original Experiment-011 probe's fixed weight vector** to (possibly ablated) activations — this measures whether the direction the probe originally relied on has been destroyed.
- `eval_refit_probe` trains a **brand-new** logistic regression on the (possibly ablated) activations — this measures whether *any* linear separability survives, i.e. whether the information moved rather than disappeared.
- `bootstrap_accuracy_ci` and `mcnemar_test` give the statistics required to say whether an observed accuracy drop is meaningful rather than noise.

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import accuracy_score, roc_auc_score
4 from scipy.stats import chi2
5
6 def project_out(H, V):
7     if V is None or V.shape[0] == 0:
8         return H
9     coords = H @ V.T
10    return H - coords @ V
11
12 def sigmoid(z):
13     return 1.0 / (1.0 + np.exp(-z))
14
15 def get_split(layer):
16     '''Reproduces Experiment 011's exact train/test split (same
17     same test_size, same X/y construction order) so frozen-prob
18     is on the identical held-out set the original probe was scc
19     benign, unsafe = load_layer_activations(layer)
20     X = np.concatenate([benign, unsafe], axis=0)
21     y = np.concatenate([np.zeros(len(benign)), np.ones(len(unsafe))], axis=0)
22     Xtr, Xte, ytr, yte = train_test_split(
23         X, y, test_size=config.test_size, random_state=config.random_state)
24     return Xtr, Xte, ytr, yte
25
26 def eval_frozen_probe(weight, bias, X, y):
27     z = X @ weight + bias

```

```

28     proba = sigmoid(z)
29     pred = (proba >= 0.5).astype(int)
30     acc = accuracy_score(y, pred)
31     auc = roc_auc_score(y, proba) if len(set(y)) > 1 else float
32     return acc, auc, pred, proba
33
34 def eval_refit_probe(Xtr, ytr, Xte, yte):
35     clf = LogisticRegression(C=config.probe_C, max_iter=config.
36     clf.fit(Xtr, ytr)
37     pred = clf.predict(Xte)
38     proba = clf.predict_proba(Xte)[: , 1]
39     acc = accuracy_score(yte, pred)
40     auc = roc_auc_score(yte, proba)
41     cleanup(clf)
42     return acc, auc, pred, proba
43
44 def bootstrap_accuracy_ci(correct_binary, n_boot=None, seed=0):
45     n_boot = n_boot or config.n_bootstrap
46     rng = np.random.default_rng(seed)
47     n = len(correct_binary)
48     means = np.empty(n_boot)
49     for i in range(n_boot):
50         idx = rng.integers(0, n, n)
51         means[i] = correct_binary[idx].mean()
52     lo, hi = np.percentile(means, [(1 - config.ci_level) / 2 *
53     return float(correct_binary.mean()), float(lo), float(hi)
54
55 def mcnemar_test(baseline_correct, ablated_correct):
56     '''Paired test on the same held-out instances: did ablation
57     instances the probe got right, more than chance?'''
58     b = int(np.sum(baseline_correct & ~ablated_correct)) # cc
59     c = int(np.sum(~baseline_correct & ablated_correct)) # in
60     if b + c == 0:
61         stat, p = 0.0, 1.0
62     else:
63         stat = (abs(b - c) - 1) ** 2 / (b + c) # continuity-c
64         p = float(1 - chi2.cdf(stat, 1))
65     return {"b_correct_to_incorrect": b, "c_incorrect_to_correc
66         "statistic": float(stat), "p_value": p}
67
68 def cohens_h(p1, p2):
69     '''Effect size for the difference between two proportions (
70     return float(2 * np.arcsin(np.sqrt(np.clip(p1, 0, 1))) - 2
71
72 def random_orthonormal_subspace(hidden_size, k, rng):
73     A = rng.normal(size=(hidden_size, k))
74     Q, _ = np.linalg.qr(A)
75     return Q.T # [k, hidden_size], rows orthonormal

```

✓ 6. Ablation sweep

For every `k` in `config.k_list`, and for every representative layer and subspace type, we ablate the top- k components, evaluate frozen + re-fit probes, run the random-subspace control, and **save** `results_k{k}.csv` **immediately** — matching the incremental-save convention from Experiment 011. Nothing generation-related happens here; the arrays involved are all small ($\leq$ a few hundred rows $\times$ 1536 dims), so memory pressure is minimal, but we still `del` + `gc.collect()` per k as a matter of house style.

```

1 ABLATION_STAGE = "policy_subspace_ablation_sweep"
2
3 def run_ablation_for_k(k):
4     rows = []
5     for layer_name, layer in rep_layers.items():
6         Xtr, Xte, ytr, yte = get_split(layer)
7         probe = load_pickle(probe_path(layer))
8         w, b = probe["weight"], probe["bias"]
9
10        # unablated baseline (k=0), needed for McNemar pairing
11        base_acc, base_auc, base_pred, base_proba = eval_froze
12        base_correct = (base_pred == yte)
13
14        for subspace_type in config.subspace_types:
15            pca = load_pickle(within_pca_path(layer) if subspa
16            V = pca["components"][:k]
17
18            Xtr_abl = project_out(Xtr, V)
19            Xte_abl = project_out(Xte, V)
20
21            f_acc, f_auc, f_pred, f_proba = eval_frozen_probe(
22            f_correct = (f_pred == yte)
23            f_mean, f_lo, f_hi = bootstrap_accuracy_ci(f_corre
24            f_mcnemar = mcnemar_test(base_correct, f_correct)
25            f_effect = cohens_h(f_acc, base_acc)
26
27            r_acc, r_auc, r_pred, r_proba = eval_refit_probe(x
28            r_correct = (r_pred == yte)
29            r_mean, r_lo, r_hi = bootstrap_accuracy_ci(r_corre
30            r_mcnemar = mcnemar_test(base_correct, r_correct)
31            r_effect = cohens_h(r_acc, base_acc)

```

```

32
33         rows.append({
34             "k": k, "layer_name": layer_name, "layer": lay
35             "eval_type": "frozen_probe", "accuracy": f_acc
36             "ci_lo": f_lo, "ci_hi": f_hi, "baseline_accura
37             "mcnemar_p": f_mcnemar["p_value"], "cohens_h":
38             "n_test": len(yte),
39         })
40         rows.append({
41             "k": k, "layer_name": layer_name, "layer": lay
42             "eval_type": "refit_probe", "accuracy": r_acc,
43             "ci_lo": r_lo, "ci_hi": r_hi, "baseline_accura
44             "mcnemar_p": r_mcnemar["p_value"], "cohens_h":
45             "n_test": len(yte),
46         })
47         cleanup(Xtr_abl, Xte_abl)
48
49     # random-subspace control (k-dim, same layer, n_ran
50     rng = np.random.default_rng(config.seed + 1000 * k + 1
51     f_accs, r_accs = [], []
52     hidden_size = Xtr.shape[1]
53     for t in range(config.n_random_trials):
54         V_rand = random_orthonormal_subspace(hidden_size,
55         Xte_r = project_out(Xte, V_rand)
56         Xtr_r = project_out(Xtr, V_rand)
57         f_acc_r, _, _, _ = eval_frozen_probe(w, b, Xte_r,
58         r_acc_r, _, _, _ = eval_refit_probe(Xtr_r, ytr, Xt
59         f_accs.append(f_acc_r)
60         r_accs.append(r_acc_r)
61         cleanup(Xte_r, Xtr_r)
62     rows.append({
63         "k": k, "layer_name": layer_name, "layer": layer,
64         "eval_type": "frozen_probe", "accuracy": float(np.
65         "ci_lo": float(np.percentile(f_accs, 2.5)), "ci_hi
66         "baseline_accuracy": base_acc, "mcnemar_p": float(
67         "cohens_h": cohens_h(float(np.mean(f_accs)), base_
68     })
69     rows.append({
70         "k": k, "layer_name": layer_name, "layer": layer,
71         "eval_type": "refit_probe", "accuracy": float(np.r
72         "ci_lo": float(np.percentile(r_accs, 2.5)), "ci_hi
73         "baseline_accuracy": base_acc, "mcnemar_p": float(
74         "cohens_h": cohens_h(float(np.mean(r_accs)), base_
75     })
76
77     cleanup(Xtr, Xte, ytr, yte, probe)
78
79     return rows

```

```

80
81 import pandas as pd
82 from tqdm.auto import tqdm
83
84 if stage_done(ABLATION_STAGE):
85     print("Ablation sweep already complete – loading cached pe
86     all_rows_df = pd.concat([pd.read_csv(os.path.join(ABLATION
87                             ignore_index=True)
88 else:
89     per_k_dfs = []
90     for k in tqdm(config.k_list, desc="ablation sweep over k")
91         out_path = os.path.join(ABLATION_DIR, f"results_k{k}.c
92         if os.path.exists(out_path):
93             df_k = pd.read_csv(out_path)
94         else:
95             rows = run_ablation_for_k(k)
96             df_k = pd.DataFrame(rows)
97             df_k.to_csv(out_path, index=False) # save immedia
98             print(f"Saved {out_path} ({len(df_k)} rows)")
99             per_k_dfs.append(df_k)
100         gc.collect()
101     all_rows_df = pd.concat(per_k_dfs, ignore_index=True)
102     mark_done(ABLATION_STAGE, meta={"k_list": config.k_list, "
103
104 all_rows_df.to_csv(os.path.join(ABLATION_DIR, "results_all_k.c
105 all_rows_df.head(12)
106

```

ablation sweep over k: 100%

5/5 [04:25<00:00, 48.25s/it]

Saved /content/nps_exp012_policy_subspace_intervention/results/ablat
 Saved /content/nps_exp012_policy_subspace_intervention/results/ablat
 Saved /content/nps_exp012_policy_subspace_intervention/results/ablat
 Saved /content/nps_exp012_policy_subspace_intervention/results/ablat
 Saved /content/nps_exp012_policy_subspace_intervention/results/ablat
 [stage] 'policy_subspace_ablation_sweep' marked complete.

	k	layer_name	layer	subspace_type	eval_type	accuracy	auc
0	1	early	4	within	frozen_probe	1.000	1.000000
1	1	early	4	within	refit_probe	1.000	1.000000
2	1	early	4	between	frozen_probe	0.950	1.000000
3	1	early	4	between	refit_probe	1.000	1.000000
4	1	early	4	random_control	frozen_probe	1.000	NaN
5	1	early	4	random_control	refit_probe	1.000	NaN
6	1	middle	14	within	frozen_probe	0.500	0.651250

7	1	middle	14	within	refit_probe	0.475	0.406875
8	1	middle	14	between	frozen_probe	0.500	0.531250
9	1	middle	14	between	refit_probe	0.275	0.195000
10	1	middle	14	random_control	frozen_probe	1.000	NaN
11	1	middle	14	random_control	refit_probe	1.000	NaN

✓ 7. Aggregate statistics

Writes `statistics.json` summarizing, per layer and subspace type, the accuracy drop from baseline, bootstrap CIs, McNemar significance, and effect size, plus a comparison against the random-subspace control (a subspace-specific effect should exceed what removing a random k -dim subspace does).

```

1 STATS_STAGE = "policy_subspace_ablation_statistics"
2
3 def build_statistics(df):
4     stats = {}
5     for layer_name, layer in rep_layers.items():
6         stats[layer_name] = {"layer": layer, "by_k": {}}
7         for k in config.k_list:
8             sub = df[(df.layer_name == layer_name) & (df.k == k)]
9             entry = {}
10            for subspace_type in config.subspace_types + ["rand":
11                s2 = sub[sub.subspace_type == subspace_type]
12                entry[subspace_type] = {
13                    row.eval_type: {
14                        "accuracy": row.accuracy, "auc": row.au
15                        "ci_lo": row.ci_lo, "ci_hi": row.ci_hi,
16                        "baseline_accuracy": row.baseline_accur
17                        "accuracy_drop": row.baseline_accuracy
18                        "mcnemar_p": row.mcnemar_p, "cohens_h":
19                    } for row in s2.itertuples()
20                }
21            stats[layer_name]["by_k"][str(k)] = entry
22    return stats
23
24 if stage_done(STATS_STAGE):
25     with open(os.path.join(STATS_DIR, "statistics.json")) as f:
26         statistics = json.load(f)
27     print("Statistics already computed – loaded from cache.")
28 else:

```



```

29     statistics = build_statistics(all_rows_df)
30     with open(os.path.join(STATS_DIR, "statistics.json"), "w")
31         json.dump(statistics, f, indent=2, default=float)
32     mark_done(STATS_STAGE)
33
34 print(json.dumps(statistics["middle"]["by_k"].get(str(config.k_
35

```

```

        accuracy : 0.5,
        "auc": 0.5618749999999999,
        "ci_lo": 0.3875,
        "ci_hi": 0.6125,
        "baseline_accuracy": 1.0,
        "accuracy_drop": 0.5,
        "mcnemar_p": 6.984393063902417e-10,
        "cohens_h": -1.5707963267948963
    },
    "refit_probe": {
        "accuracy": 0.375,
        "auc": 0.305625,
        "ci_lo": 0.275,
        "ci_hi": 0.475,
        "baseline_accuracy": 1.0,
        "accuracy_drop": 0.625,
        "mcnemar_p": 4.218958515878057e-12,
        "cohens_h": -1.8234765819369751
    }
},
    "between": {
        "frozen_probe": {
            "accuracy": 0.5,
            "auc": 0.496875,
            "ci_lo": 0.3875,
            "ci_hi": 0.6125,
            "baseline_accuracy": 1.0,
            "accuracy_drop": 0.5,
            "mcnemar_p": 6.984393063902417e-10,
            "cohens_h": -1.5707963267948963
        },
        "refit_probe": {
            "accuracy": 0.2375,
            "auc": 0.11312500000000002,
            "ci_lo": 0.15,
            "ci_hi": 0.32531249999999983,
            "baseline_accuracy": 1.0,
            "accuracy_drop": 0.7625,
            "mcnemar_p": 1.5654144647214707e-14,
            "cohens_h": -2.1235114398916797
        }
    },
    "random_control": {
        "frozen_probe": {
            "accuracy": 1.0,

```

```

    "auc": NaN,
    "ci_lo": 1.0,
    "ci_hi": 1.0,
    "baseline_accuracy": 1.0,
    "accuracy_drop": 0.0,
    "mcnemar_p": NaN,
    "cohens_h": 0.0
  },
  "refit_probe": {
    "accuracy": 1.0,
    "auc": NaN,
    "ci_lo": 1.0,
    "ci_hi": 1.0,
    "baseline_accuracy": 1.0,

```

✓ 8. Publication-quality figures

Saved as `.png` (300 dpi) and `.pdf` under `results/figures/`, matching the Experiment 011 convention.

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 sns.set_theme(style="whitegrid", context="paper", font_scale=1.
5
6 def save_fig(fig, name):
7     fig.savefig(os.path.join(FIGURES_DIR, f"{name}.png"), dpi=3
8     fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_
9     plt.close(fig)
10    print("Saved figure:", name)
11
12 layer_order = list(rep_layers.keys())
13 layer_colors = dict(zip(layer_order, sns.color_palette("viridis
14

```

```

1 # Fig 1: frozen-probe accuracy vs k removed, one panel per subs
2 # one line per layer, random-control band shown for comparison.
3 fig, axes = plt.subplots(1, len(config.subspace_types), figsize
4 if len(config.subspace_types) == 1:
5     axes = [axes]
6 for ax, subspace_type in zip(axes, config.subspace_types):
7     for layer_name in layer_order:
8         sub = all_rows_df[(all_rows_df.layer_name == layer_name
9                             (all_rows_df.subspace_type == subspa
10                             (all_rows_df.eval_type == "frozen_pr
11         ax.plot(sub.k, sub.accuracy, marker="o", label=f"{layer
12                 color=layer_colors[layer_name])
13         ax.fill_between(sub.k, sub.ci_lo, sub.ci_hi, alpha=0.15
14         rand = all_rows_df[(all_rows_df.layer_name == layer_nar
15                             (all_rows_df.subspace_type == "rand
16                             (all_rows_df.eval_type == "frozen_p
17         ax.plot(rand.k, rand.accuracy, linestyle="--", color=la
18         ax.set_title(f"{subspace_type}-class subspace")
19         ax.set_xlabel("Principal components removed (k)")
20         ax.axhline(0.5, color="gray", linewidth=0.8, linestyle=":")
21 axes[0].set_ylabel("Frozen-probe accuracy\n(original Exp. 011 w
22 axes[-1].legend(fontsize=8, loc="lower left")
23 fig.suptitle("Does removing the policy subspace break the ORIGI
24 save_fig(fig, "fig1_frozen_probe_accuracy_vs_k")
25

```

```

/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
Saved figure: fig1_frozen_probe_accuracy_vs_k

```

```

1 # Fig 2: re-fit probe accuracy vs k removed – tests whether the
2 # is destroyed or just moved elsewhere in the representation.
3 fig, axes = plt.subplots(1, len(config.subspace_types), figsize
4 if len(config.subspace_types) == 1:
5     axes = [axes]
6 for ax, subspace_type in zip(axes, config.subspace_types):
7     for layer_name in layer_order:
8         sub = all_rows_df[(all_rows_df.layer_name == layer_name
9                             (all_rows_df.subspace_type == subspa
10                             (all_rows_df.eval_type == "refit_prc
11         ax.plot(sub.k, sub.accuracy, marker="s", label=f"{layer
12                 color=layer_colors[layer_name])
13         ax.fill_between(sub.k, sub.ci_lo, sub.ci_hi, alpha=0.15
14         rand = all_rows_df[(all_rows_df.layer_name == layer_nar
15                             (all_rows_df.subspace_type == "rand
16                             (all_rows_df.eval_type == "refit_pr
17         ax.plot(rand.k, rand.accuracy, linestyle="--", color=la
18         ax.set_title(f"{subspace_type}-class subspace")
19         ax.set_xlabel("Principal components removed (k)")
20         ax.axhline(0.5, color="gray", linewidth=0.8, linestyle=":")
21 axes[0].set_ylabel("Re-fit probe accuracy\n(new classifier trai
22 axes[-1].legend(fontsize=8, loc="lower left")
23 fig.suptitle("Does ANY linear separability survive after the pc
24 save_fig(fig, "fig2_refit_probe_accuracy_vs_k")
25

```

```

/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
Saved figure: fig2_refit_probe_accuracy_vs_k

```

```

1 # Fig 3: combined trade-off – frozen vs. refit accuracy, within
2 # middle layer as the representative case (redundancy vs. genui
3 fig, ax = plt.subplots(figsize=(7, 4.5))
4 for eval_type, marker, color in [("frozen_probe", "o", "tab:blu
5     sub = all_rows_df[(all_rows_df.layer_name == "middle") &
6                         (all_rows_df.subspace_type == "within")
7                         (all_rows_df.eval_type == eval_type)].sc
8     ax.plot(sub.k, sub.accuracy, marker=marker, color=color,
9             label="Frozen probe" if eval_type == "frozen_probe"
10    ax.fill_between(sub.k, sub.ci_lo, sub.ci_hi, alpha=0.15, cc
11 ax.axhline(0.5, color="gray", linewidth=0.8, linestyle=":")
12 ax.set_xlabel("Principal components removed (k)")
13 ax.set_ylabel("Accuracy")
14 ax.set_title(f"Frozen vs. re-fit probe accuracy – middle layer
15             "Large gap => information is redundant/re-encoded,
16 ax.legend()
17 save_fig(fig, "fig3_combined_tradeoff_frozen_vs_refit")
18

```

Saved figure: fig3_combined_tradeoff_frozen_vs_refit
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=

```

1 # Fig 4: layer x k heatmap of accuracy drop (baseline – ablated
2 # within-class subspace.
3 sub = all_rows_df[(all_rows_df.subspace_type == "within") & (al
4 sub["accuracy_drop"] = sub["baseline_accuracy"] – sub["accuracy
5 pivot = sub.pivot(index="layer_name", columns="k", values="accu
6
7 fig, ax = plt.subplots(figsize=(6.5, 3.5))
8 sns.heatmap(pivot, annot=True, fmt=".3f", cmap="rocket_r", ax=a
9 ax.set_xlabel("Principal components removed (k)")
10 ax.set_ylabel("Layer")
11 ax.set_title("Frozen-probe accuracy drop after within-class sub
12 save_fig(fig, "fig4_layer_by_k_heatmap_frozen_within")
13

```

Saved figure: fig4_layer_by_k_heatmap_frozen_within
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=

```

1 # Fig 5: within- vs. between-class subspace comparison at a fix
2 # for both eval types – a compact summary panel.
3 k_fixed = config.k_list[len(config.k_list) // 2] # e.g. k=5 wi
4 sub = all_rows_df[all_rows_df.k == k_fixed]
5
6 fig, ax = plt.subplots(figsize=(8, 4.5))
7 width = 0.18
8 x = np.arange(len(layer_order))
9 offsets = {("within", "frozen_probe"): -1.5, ("within", "refit_
10           ("between", "frozen_probe"): 0.5, ("between", "refit
11 colors = {("within", "frozen_probe"): "#4C72B0", ("within", "re
12           ("between", "frozen_probe"): "#DD8452", ("between", "
13 for (subspace_type, eval_type), off in offsets.items():
14     vals = [sub[(sub.layer_name == l) & (sub.subspace_type == s
15             (sub.eval_type == eval_type)][("accuracy").value
16     ax.bar(x + off * width, vals, width,
17           label=f"{subspace_type}-class / {eval_type.replace('
18           color=colors[(subspace_type, eval_type)])
19 ax.axhline(0.5, color="gray", linewidth=0.8, linestyle=":")
20 ax.set_xticks(x)
21 ax.set_xticklabels([f"{l}\n(L{rep_layers[l]})" for l in layer_c
22 ax.set_ylabel("Accuracy")
23 ax.set_title(f"Ablation comparison at k={k_fixed} components re
24 ax.legend(fontsize=8, ncol=2)
25 save_fig(fig, "fig5_within_vs_between_comparison_fixed_k")
26
27 print("All figures written to:", FIGURES_DIR)
28

```

```

Saved figure: fig5_within_vs_between_comparison_fixed_k
All figures written to: /content/nps_exp012_policy_subspace_interver
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=
/tmp/ipykernel_1101/366223031.py:8: MatplotlibDeprecationWarning: Th
fig.savefig(os.path.join(FIGURES_DIR, f"{name}.pdf"), bbox_inches=

```

✓ 9. Auto-generated report

Builds `REPORT.md` with the structure requested for this experiment: Research Question, Methodology, Experimental Setup, Results, Figures, Interpretation, Discussion, Limitations, Future Work. The report only states what the ablation-on-probes experiment actually measured — it does not extrapolate to generation-time behavior, which this notebook never tests.

```

1 def get_metric(layer_name, k, subspace_type, eval_type, field):
2     row = all_rows_df[(all_rows_df.layer_name == layer_name) &
3                       (all_rows_df.subspace_type == subspace_t
4     return row.iloc[0][field] if len(row) else float("nan")
5
6 k_max = max(config.k_list)
7 report = []
8 report.append("# NPS Experiment 012 – Policy Subspace Causal Ne
9 report.append("## Research Question\n")
10 report.append(
11     "Is the policy subspace discovered in Experiment 011 causal
12     "*linear separability* of benign vs. unsafe activations, or
13     "redundantly encoded elsewhere in the residual stream? This
14     "generation-time behavior (refusal, jailbreak susceptibility
15     "on activations with the discovered subspace projected out.
16 )
17
18 report.append("## Methodology\n")
19 report.append(
20     f"For each representative layer (early=L{rep_layers['early']
21     f"final=L{rep_layers['final']}) and each subspace definitio
22     f"(within-class PCA, between-class paired-difference PCA),
23     f"(k ∈ {config.k_list}) were projected out of held-out beni
24     f"(test split reproduced with Experiment 011's exact seed/t
25     f"were run on the ablated activations: (a) the **original,
26     f"probe weight vector, and (b) a **freshly re-fit** logisti
27     f"ablated training split. Both were compared against ablati
28     f"subspace ({config.n_random_trials} trials, averaged) as a
29 )
30
31 report.append("## Experimental Setup\n")
32 report.append(f"- Model: `{config.model_name}` (activations reu
33 report.append(f"- k values: {config.k_list}")
34 report.append(f"- Subspace types: {config.subspace_types} + ran
35 report.append(f"- Bootstrap resamples for CIs: {config.n_bootst
36 report.append(f"- Random-subspace control trials: {config.n_ran
37 report.append(f"- Test split: seed={config.seed}, test_size={co
38
39 report.append("## Results\n")
40 report.append("### Frozen-probe accuracy (original Exp. 011 dec
41 report.append("| Layer | Subspace | k=" + " | k=".join(str(k) f
42 report.append("|---|---|" + "---|" * len(config.k_list))
43 for layer_name in layer_order:
44     for subspace_type in config.subspace_types + ["random_contr
45         vals = [f"{get_metric(layer_name, k, subspace_type, 'fr
46         report.append(f"| {layer_name} (L{rep_layers[layer_name
47 report.append("")
48

```

```

49 report.append("### Re-fit probe accuracy (new classifier on abl
50 report.append("| Layer | Subspace | k=" + " | k=".join(str(k) f
51 report.append("|---|---|" + "---|" * len(config.k_list))
52 for layer_name in layer_order:
53     for subspace_type in config.subspace_types + ["random_contr
54         vals = [f"{get_metric(layer_name, k, subspace_type, 're
55         report.append(f"| {layer_name} | {rep_layers[layer_name]
56 report.append("")
57
58 # headline pattern check: does frozen drop while refit stays hi
59 # or do both drop together (genuine necessity)?
60 mid_frozen_kmax = get_metric("middle", k_max, "within", "frozen
61 mid_refit_kmax = get_metric("middle", k_max, "within", "refit_
62 mid_base = get_metric("middle", config.k_list[0], "withi
63 gap = mid_refit_kmax - mid_frozen_kmax
64
65 report.append("## Interpretation\n")
66 report.append(
67     f"At the middle representative layer, within-class subspace
68     f"subspace (k={k_max}) leaves frozen-probe accuracy at **{m
69     f"(baseline {mid_base:.3f}) while a freshly re-fit probe on
70     f"reaches **{mid_refit_kmax:.3f}** – a gap of **{gap:+.3f}*
71     + ("A large positive gap indicates the class-relevant infor
72         "encoded*: the specific direction the original probe rel
73         "policy-relevant information remains linearly recoverabl
74         "stream at this layer. This means the discovered subspac
75         "the information, even though it is the direction the tr
76         if gap > 0.1 else
77         "A small gap indicates that once this subspace is remove
78         "linear signal remains – consistent with the discovered
79         "causally necessary for the *linear* representation of p
80         "layer (within the scope of what this notebook tests).")
81     + "\n"
82 )
83 report.append(
84     "Compare each subspace-ablation row against its `random_con
85     "if random k-dimensional ablation causes a similar accuracy
86     "generic consequence of removing *any* k dimensions from a
87     "not evidence that this particular subspace is special. See
88     "McNemar p-values and Cohen's h effect sizes quantifying th
89 )
90
91 report.append("## Figures\n")
92 for fig_name, caption in [
93     ("fig1_frozen_probe_accuracy_vs_k", "Frozen (original) prob
94     ("fig2_refit_probe_accuracy_vs_k", "Re-fit probe accuracy v
95     ("fig3_combined_tradeoff_frozen_vs_refit", "Frozen vs. re-f
96     ("fig4_layer_by_k_heatmap_frozen_within", "Layer x k heatma
97     ("fig5_within_vs_between_comparison_fixed_k", f"Within- vs

```



```

97     \figs_within_vs_between_comparison_fixed_n, \figs_within_vs_
98 ]:
99     report.append(f"- `figures/{fig_name}.png` - {caption}")
100 report.append("")
101
102 report.append("## Discussion\n")
103 report.append(
104     "This experiment establishes a *representational* causal cl
105     "policy subspace is necessary for linear probes to separate
106     "does not establish, and should not be read as establishing
107     "model's generation-time behavior (refusals, compliance) ca
108     "that would require an intervention during generation, whic
109     "not perform.\n"
110 )
111
112 report.append("## Limitations\n")
113 report.append(
114     "- Held-out test sets are small (governed by Experiment 011
115     "per-condition confidence intervals may be wide – consult `
116     "estimates alone.\n"
117     "- Linear probes only test *linear* separability; a nonline
118     "from ablated activations that a logistic regression cannot
119     "- Results are specific to `Qwen/Qwen2.5-1.5B-Instruct` and
120     "distribution; they may not generalize to other models or m
121     "unsafe prompts.\n"
122     "- No claim is made here about whether this subspace is the
123     "in the model – only whether it's necessary for linear sepa
124 )
125
126 report.append("## Future Work\n")
127 report.append(
128     "- If a future experiment does test generation-time behavio
129     "intervention, it should be scoped and reviewed separately
130     "consideration of dual-use risk given that the same techniq
131     "method on open-weight models.\n"
132     "- Extend the redundancy check with nonlinear probes (e.g.
133     "frozen/re-fit gap narrows or widens.\n"
134     "- Repeat across model scale (7B/14B Qwen2.5) to see whethe
135     "capacity.\n"
136 )
137
138 report_path = os.path.join(RESULTS_DIR, "REPORT.md")
139 with open(report_path, "w") as f:
140     f.write("\n".join(report))
141 print("Wrote report to:", report_path)
142 print("\n".join(report[:20]))
143

```

Wrote report to: /content/nps_exp012_policy_subspace_intervention/re

```
# NPS Experiment 012 – Policy Subspace Causal Necessity – Report

## Research Question

Is the policy subspace discovered in Experiment 011 causally necessary?

## Methodology

For each representative layer (early=L4, middle=L14, final=L25) and

## Experimental Setup

- Model: `Qwen/Qwen2.5-1.5B-Instruct` (activations reused from Experiment 011)
- k values: [1, 2, 5, 10, 20]
- Subspace types: ['within', 'between'] + random_control
- Bootstrap resamples for CIs: 2000
- Random-subspace control trials: 20
- Test split: seed=42, test_size=0.2 (matches Experiment 011)

## Results

### Frozen-probe accuracy (original Exp. 011 decision boundary)
```

Layer	Subspace	k=1	k=2	k=5	k=10	k=20
early (L4)	within	1.000	0.975	0.500	0.500	0.500
early (L4)	between	0.950	0.787	0.500	0.500	0.500
early (L4)	random_control	1.000	1.000	1.000	1.000	1.000
middle (L14)	within	0.500	0.500	0.500	0.500	0.500

10. Package results

Zips `results/` as `NPS_Experiment_012_Results.zip`, matching the requested export name.

```
1 import shutil
2
3 zip_base = os.path.join(BASE_DIR, "NPS_Experiment_012_Results")
4 zip_path = shutil.make_archive(zip_base, "zip", RESULTS_DIR)
5 print("Zipped results to:", zip_path)
6 print("Size (MB):", round(os.path.getsize(zip_path) / 1e6, 2))
7
```

```
Zipped results to: /content/nps_exp012_policy_subspace_intervention/
Size (MB): 1.03
```

11. Summary

This notebook tested whether the Experiment 011 policy subspace is causally necessary for **linear separability** of policy-relevant activations, by ablating it and checking (a) whether the original probe breaks and (b) whether a freshly trained probe can still find a separating direction — both against a random-subspace control, with bootstrap CIs and paired significance tests.

What this notebook intentionally did not do: generate text, evaluate jailbreak success, or measure refusal rate under intervention. If a genuinely behavioral follow-up is wanted later, it should be scoped and reviewed as its own experiment rather than folded into this one, given that the same mechanics (activation ablation + evaluation against unsafe prompts) is also the recipe for refusal-ablation on an open-weight model.